

# a09g-2-comm-protocols

---

- Team Number: T06
- Team Name: Byte Crafter
- Team Members: Tony Yan & Yue Zhang
- GitHub Repository URL: <https://github.com/ese5160/final-project-t06-byte-crafter>
- Description of test hardware: ROG Zephyrus G14, HUAWEI 14

## 2. Your most important (high-risk) peripheral

### 2.1 A section in your readme explaining the following points

#### 1. Current Situation

##### 1. What your high-risk component is:

The high-risk component that requires to write firmware library for this time is the IMU (Inertial Measurement Unit).

##### 2. What makes it high risk:

The IMU is considered high-risk because it plays a critical security role in this project—it needs to detect vibrations or movements caused by tampering, forced entry, or physical attacks. If the IMU fails to function correctly, the system may not detect real-world intrusions, compromising its integrity.

##### 3. How it interfaces with the MCU and the rest of the system (i.e. I2C, SPI, ADC, GPIO, etc.):

The IMU communicates with the MCU over the I2C protocol. Additionally, its INT1 pin may be used as a GPIO interrupt input to alert the MCU of detected motion events, allowing for efficient real-time response.

#### 2. Future plans

##### 1. What tests do you plan to run to verify the functionality of your peripheral and library code:

1. I<sup>2</sup>C Communication Test: Ensure correct initialization, reading and writing registers.
2. WHO\_AM\_I Register Check: Confirm the device is responding correctly over I2C.
3. Raw Data Readout: Read accelerometer X/Y/Z values and verify they're within expected ranges when stationary and during movement.
4. Interrupt Trigger Test: Configure motion or activity detection in the IMU and verify that the INT1 pin triggers an interrupt to the MCU.
5. Tamper Simulation Test: Apply vibration or shock to the IMU and check if the firmware correctly flags the event.

##### 2. Conclusions and how you plan to proceed based on the information from testing:

Based on the results of these tests:

1. If successful: Integrate IMU event handling into the main system logic.
2. If issues arise: Debug I2C timing, sensor configuration registers, or MCU interrupt setup.

##### 3. If you haven't received your hardware, how you plan to create something that you can just wire up and test as soon as the hardware arrives:

1. All IMU library API and I2C driver will be developed for firmware testing.
2. Use the IMU in the class kit for early prototyping.
3. This will allow firmware and interrupt handling code to be written and validated ahead of time, minimizing delay once real hardware is available.

## 2.2 List the directory where your library code is located in your repository readme

Here is the IMU library we wrote for our project:

[lsm6dso\\_reg.c](#)

And how to use it in main function:

[main21.c](#)